

CREACIÓN DE APIS MODERNAS CON PYTHON

Introducción a FastAPI



Objetivos de Aprendizaje

- Comprender qué es FastAPI y sus ventajas principales
- Configurar un entorno virtual e instalar FastAPI
- Crear un servidor básico con rutas GET y POST
- Implementar validación de datos con Pydantic
- Trabajar con path parameters y query parameters



¿Qué es FastAPI?



Introducción a FastAPI

- **FastAPI es un framework web moderno y rápido** para crear APIs con Python 3.7+
- Basado en Python Type Hints (anotaciones de tipo)
- Diseñado para ser intuitivo, rápido de desarrolla y listo para producción
- Creado en 2018

Características principales:

- ⚡ **Muy rápido:** Uno de los frameworks más veloces
- 🎯 **Fácil de usar:** Sintaxis simple e intuitiva
- 📖 **Documentación automática:** Genera docs interactivas automáticamente
- 🔒 **Validación automática:** Valida datos de entrada y salida

Ventajas de FastAPI

1. Rendimiento Excepcional

- Comparable a NodeJS y Go en velocidad
- Soporte nativo para programación asíncrona

2. Desarrollo Productivo

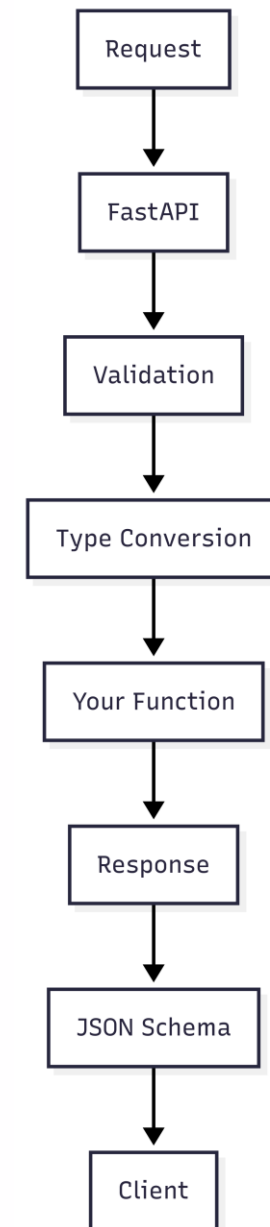
- Menos código para la misma funcionalidad
- Detección automática de errores
- Autocompletado inteligente en IDEs

3. Estándares Modernos

- Basado en OpenAPI (ant. Swagger)
- Soporte completo para JSON Schema
- Compatible con OAuth 2.0, JWT, etc.

4. Validación Automática

- Pydantic para validación de datos
- Convierte automáticamente tipos de datos

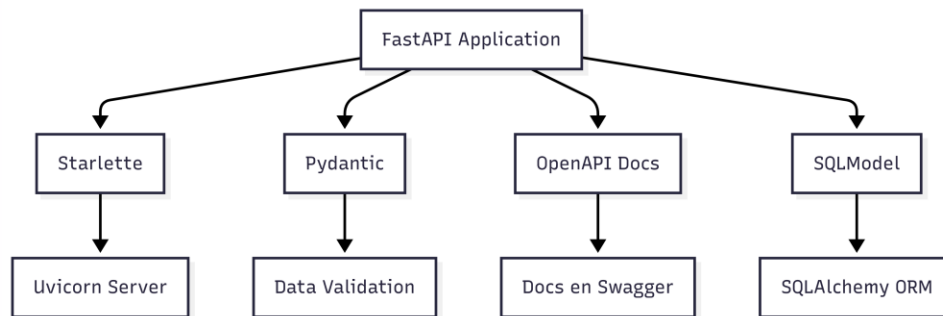


Comparativa de David con otros Frameworks

Característica	FastAPI	Flask	Django REST
Velocidad	★ ★ ★ ★ ★	★ ★ ★	★ ★
Facilidad de aprendizaje	★ ★ ★ ★	★ ★ ★ ★ ★	★ ★
Documentación automática	★ ★ ★ ★ ★	★	★ ★ ★
Validación de datos	★ ★ ★ ★ ★	★ ★	★ ★ ★ ★
Soporte async nativo	★ ★ ★ ★ ★	★ ★ ★	★ ★ ★
Type hints	★ ★ ★ ★ ★	★	★ ★



Tecnologías que Trabajan con FastAPI



Tecnologías Base:

- **Starlette:** Framework ASGI de alta performance
- **Pydantic:** Validación de datos usando Python type hints
- **Uvicorn:** Servidor ASGI para desarrollo y producción

Herramientas Complementarias:

- **SQLAlchemy:** ORM para bases de datos SQL (SQLModel está basado en esta)
- **Alembic:** Migraciones de base de datos
- **Pytest:** Testing unitario entre otros
- **Docker:** Containerización
- **OpenAPI/Swagger:** Documentación interactiva

Configuración del Entorno



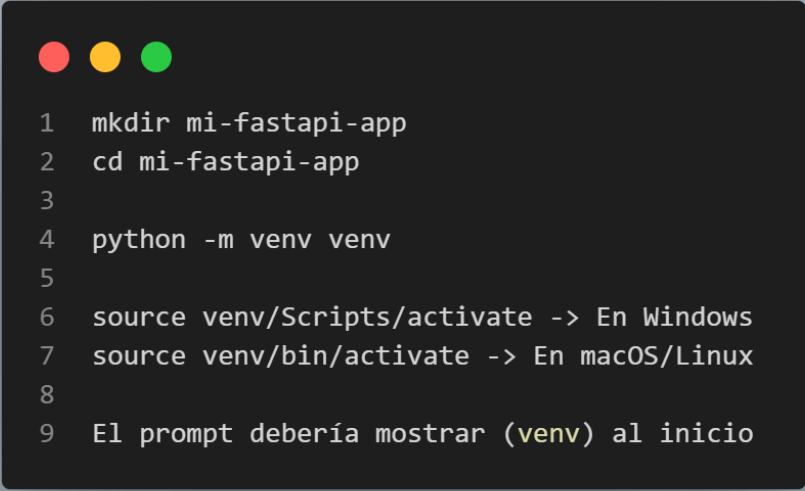
Creación del Entorno Virtual

¿Por qué usar entornos virtuales?

- Aíslan las dependencias del proyecto
- Evitan conflictos entre diferentes proyectos
- Facilitan la reproducibilidad del entorno

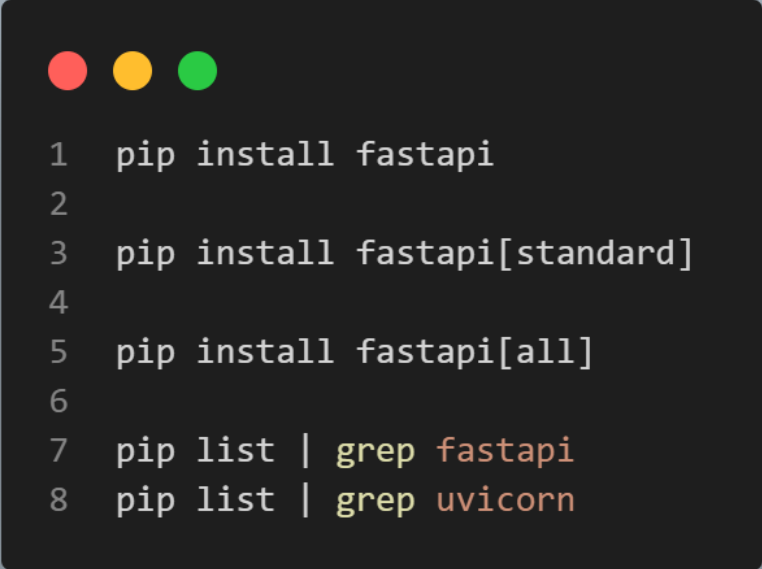
Pasos

- **Crear directorio**
- **Crear entorno virtual**
- **Activar entorno virtual**
- **Verificar activación**



```
1  mkdir mi-fastapi-app
2  cd mi-fastapi-app
3
4  python -m venv venv
5
6  source venv/Scripts/activate -> En Windows
7  source venv/bin/activate -> En macOS/Linux
8
9  El prompt debería mostrar (venv) al inicio
```

Instalación de FastAPI y Dependencias



```
1 pip install fastapi
2
3 pip install fastapi[standard]
4
5 pip install fastapi[all]
6
7 pip list | grep fastapi
8 pip list | grep uvicorn
```

- Instalación básica
- Instalación estándar (también bien)
- **Instalación completa (recomendada)**
 - Incluye
 - Uvicorn: Servidor ASGI
 - Python-multipart: Para formularios
 - Email-validator: Validación de emails
 - Requests: Cliente HTTP para testing
- Verificar instalación

"Hola Mundo" con FastAPI

1. Crear archivo 'main.py'
2. Ejecutar la aplicación
 - 'fastapi dev main.py'
3. Explicación del código
 - **'FastAPI()'**: Crea una instancia de la aplicación
 - **'@app.get("/")'**: Decorador que define una ruta GET en la raíz
 - **'async def'**: Función asíncrona (opcional pero recomendada)
 - **'return'**: FastAPI convierte automáticamente el dict a JSON

```
1 from fastapi import FastAPI
2
3 # Crear instancia de FastAPI
4 app = FastAPI()
5
6 # Definir ruta raíz
7 @app.get("/")
8 async def root():
9     return {"message": "¡Hola Mundo desde FastAPI!"}
```

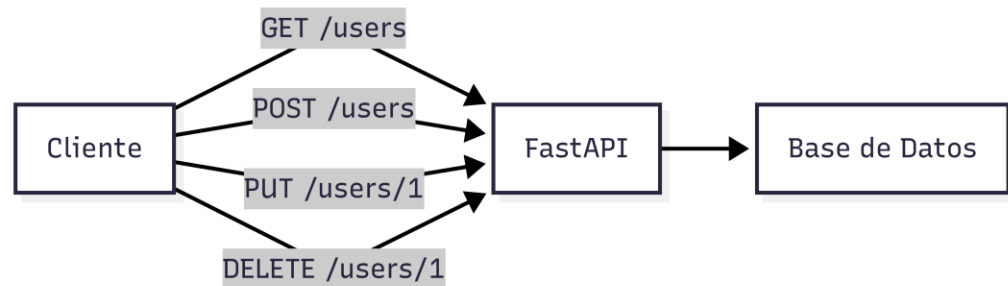
Rutas Básicas y Métodos HTTP



Rutas y Métodos HTTP

- ¿Qué son las rutas?
 - Puntos de entrada de tu API
 - Definen qué hacer cuando alguien accede a una URL específica
 - Se asocian con métodos HTTP específicos
- Métodos HTTP principales:
 - **GET**: Obtener datos
 - **POST**: Crear nuevos datos
 - **PUT**: Actualizar datos completos
 - **PATCH**: Actualizar datos parcialmente
 - **DELETE**: Eliminar datos

```
1 @app.get("/ruta")           # Para obtener datos
2 @app.post("/ruta")          # Para crear datos
3 @app.put("/ruta")           # Para actualizar datos
4 @app.delete("/ruta")        # Para eliminar datos
```



Implementando Rutas GET

```
1 from fastapi import FastAPI
2
3 app = FastAPI()
4
5 @app.get("/")
6 async def root():
7     return {"message": "Página principal"}
8
9 @app.get("/usuarios")
10 async def obtener_usuarios():
11     return {
12         "usuarios": [
13             {"id": 1, "nombre": "Ana"},
14             {"id": 2, "nombre": "Carlos"}
15         ]
16     }
17
18 @app.get("/productos")
19 async def obtener_productos():
20     return {
21         "productos": [
22             {"id": 1, "nombre": "Laptop", "precio": 999.99},
23             {"id": 2, "nombre": "Mouse", "precio": 29.99}
24         ]
25     }
```

- Usar nombres descriptivos para las funciones
- Usar sustantivos en plural para colecciones
- Retornar datos en formato JSON consistente

Implementando Rutas POST

```
1 @app.post("/usuarios")
2 async def crear_usuario():
3     # Por ahora, simplemente confirmamos la creación
4     return {"message": "Usuario creado exitosamente"}
5
6 @app.post("/productos")
7 async def crear_producto():
8     return {"message": "Producto creado exitosamente"}
```

- **¿Qué hace una ruta POST?**
 - Recibe datos del cliente
 - Procesa/valida los datos
 - Normalmente crea un nuevo recurso
 - Retorna confirmación o el recurso creado
- **Códigos de estado HTTP comunes:**
 - **200:** OK (éxito general)
 - **201:** Created (recurso creado)
 - **400:** Bad Request (datos inválidos)
 - **404:** Not Found (recurso no encontrado)
 - **500:** Internal Server Error

Organizando tu Aplicación FastAPI

```
1 from fastapi import FastAPI
2
3 app = FastAPI(
4     title="Mi API",
5     description="Una API de ejemplo con FastAPI",
6     version="1.0.0"
7 )
8
9 # Rutas de usuarios
10 @app.get("/usuarios")
11 async def obtener_usuarios():
12     return {"usuarios": []}
13
14 @app.post("/usuarios")
15 async def crear_usuario():
16     return {"message": "Usuario creado"}
17
18 # Rutas de productos
19 @app.get("/productos")
20 async def obtener_productos():
21     return {"productos": []}
22
23 @app.post("/productos")
24 async def crear_producto():
25     return {"message": "Producto creado"}
```

- Ventajas de la organización
 - Código más legible y mantenible
 - Fácil de encontrar rutas específicas
 - Mejor documentación automática

Modelos Pydantic para Validación



¿Qué es Pydantic?

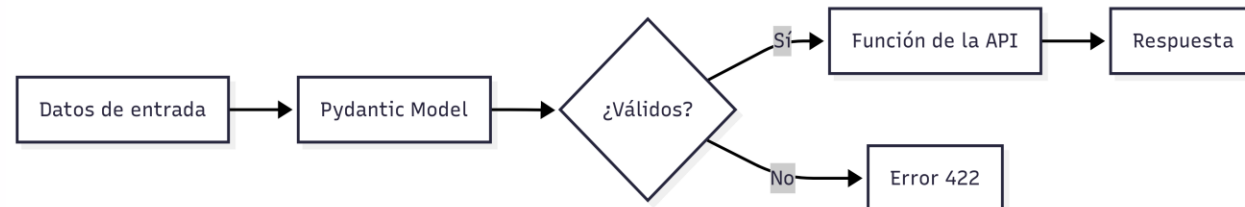
- **Pydantic es una librería de python que:**

- Valida datos usando Python type hints
- Convierte automáticamente tipos de datos
- Genera esquemas JSON automáticamente

- **¿Por qué es importante la validación?**

- **Seguridad:** Evita datos maliciosos o incorrectos
- **Consistencia:** Garantiza que los datos tengan el formato esperado
- **Documentación:** Documenta automáticamente qué datos espera tu API
- **Desarrollo:** Detecta errores temprano en el desarrollo

```
1 from pydantic import BaseModel
2
3 class Usuario(BaseModel):
4     nombre: str
5     edad: int
6     email: str
```



Definiendo Modelos de Datos

- **Tipos de datos comunes:**
 - 'str': Cadenas de texto
 - 'int': Números enteros
 - 'float': Números decimales
 - 'bool': Verdadero/Falso
 - 'List[str]': Lista de cadenas
 - 'Optional[str]': Campo opcional
 - 'datetime': Fechas y horas

```
1 from pydantic import BaseModel
2 from typing import Optional
3
4 class Usuario(BaseModel):
5     nombre: str
6     edad: int
7     email: str
8     activo: bool = True # Valor por defecto
9     telefono: Optional[str] = None # Campo opcional
10
11
12 # Esto funciona
13 usuario_valido = Usuario(
14     nombre="Ana García",
15     edad=25,
16     email="ana@email.com"
17 )
18
19 # Esto dará error (edad como string)
20 usuario_invalido = Usuario(
21     nombre="Carlos",
22     edad="veinticinco", # Error!
23     email
```

Validaciones Personalizadas con Field

```
1 class Usuario(BaseModel):
2     nombre: str = Field(
3         min_length=2,
4         max_length=50,
5         description="Nombre completo del usuario"
6     )
7     edad: int = Field(
8         ge=0, # >= 0 (greater or equal)
9         le=120, # <= 120 (less or equal)
10        description="Edad en años"
11    )
12    email: str = Field(
13        regex=r'^[\w\.-]+@[\w\.-]+\.\w+$',
14        description="Email válido"
15    )
16    salario: float = Field(
17        gt=0, # > 0 (greater than)
18        description="Salario debe ser mayor a 0"
19    )
```

- **Validaciones disponibles:**

- 'min_length' / 'max_length': Longitud de strings
- 'ge' / 'gt': Mayor o igual / Mayor que
- 'le' / 'lt': Menor o igual / Menor que
- 'regex': Expresiones regulares
- 'description': Documentación del campo

Usando Modelos en Rutas FastAPI

```
1 from fastapi import FastAPI
2 from pydantic import BaseModel, Field
3
4 app = FastAPI()
5
6 class Usuario(BaseModel):
7     nombre: str = Field(min_length=2, max_length=50)
8     edad: int = Field(ge=0, le=120)
9     email: str
10    activo: bool = True
11
12 # Usar el modelo en POST:
13 @app.post("/usuarios")
14 async def crear_usuario(usuario: Usuario):
15     # FastAPI automáticamente valida los datos
16     return {
17         "message": "Usuario creado exitosamente",
18         "usuario": usuario
19     }
```

¿Qué sucede automáticamente?

1. FastAPI lee el JSON del request body
2. Pydantic valida cada campo según las reglas
3. Si hay errores, devuelve error 422 automáticamente
4. Si está válido, convierte los datos al modelo
5. Tu función recibe el objeto validado

Definiendo Modelos para Respuestas

```
1 # Modelo para crear usuario (input):
2 class UsuarioCreate(BaseModel):
3     nombre: str = Field(min_length=2, max_length=50)
4     edad: int = Field(ge=0, le=120)
5     email: str
6
7 # Modelo para respuesta (output):
8 class UsuarioResponse(BaseModel):
9     id: int
10    nombre: str
11    edad: int
12    email: str
13    activo: bool
14    fecha_creacion: str
```

```
1 # Usar ambos modelos:
2 @app.post("/usuarios", response_model=UsuarioResponse)
3 async def crear_usuario(usuario: UsuarioCreate):
4     # Simular creación en base de datos
5     nuevo_usuario = {
6         "id": 1,
7         "nombre": usuario.nombre,
8         "edad": usuario.edad,
9         "email": usuario.email,
10        "activo": True,
11        "fecha_creacion": "2024-01-15T10:30:00"
12    }
13    return nuevo_usuario
```

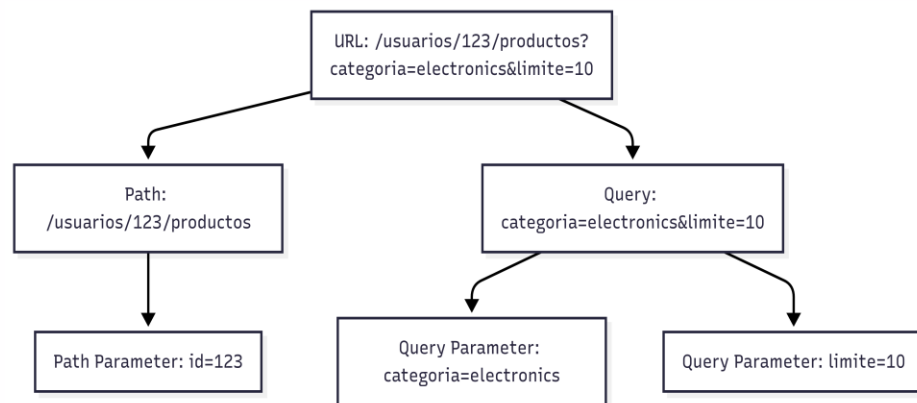
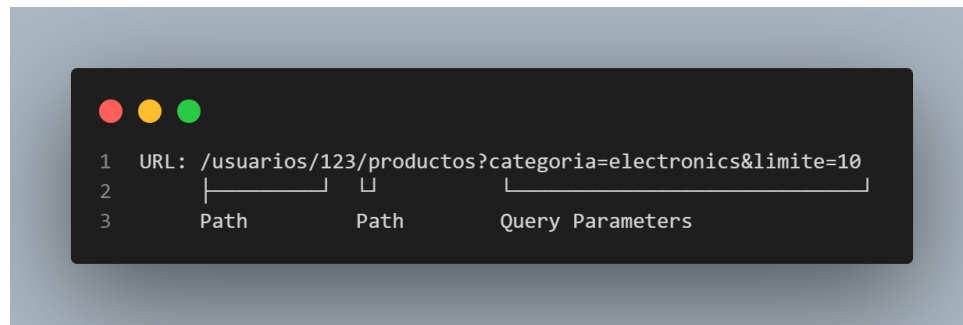
Recordatorio: Hacer mini ejemplo concreto



Path Parameters y Query Parameters



Tipos de Parámetros en FastAPI



- **Path Parameters (Parámetros de ruta)**

- Forman parte de la URL
- Son obligatorios
- Se definen en la ruta con '{}'
- Ejemplo: '/usuarios/{id}' donde 'id' es un path parameter

- **Query Parameters (Parámetros de consulta)**

- Van después del '?' En la URL
- Son opcionales (pueden tener valores por defecto)
- Se separan con '&'
- Ejemplo: '/productos?categoria=electronics&precio_max=100'

Trabajando con Path Parameters

- Validación automática:
 - FastAPI convierte automáticamente el tipo
 - Si no puede convertir, devuelve error 422
 - Ejemplo: '/usuarios/abc' -> Error si esperamos 'int'

```
1 from fastapi import Path
2
3 @app.get("/usuarios/{id}")
4 async def obtener_usuario(
5     id: int = Path(ge=1, description="ID del usuario")
6 ):
7     return {"usuario_id": id}
```

```
1 # Ejemplo básico:
2 @app.get("/usuarios/{id}")
3 async def obtener_usuario(id: int):
4     return {"usuario_id": id, "nombre": "Usuario ejemplo"}
5
6 # Múltiples path parameters:
7 @app.get("/usuarios/{user_id}/productos/{product_id}")
8 async def obtener_producto_usuario(user_id: int, product_id: int):
9     return {
10         "user_id": user_id,
11         "product_id": product_id,
12         "producto": "Laptop Dell"
13     }
```

Path Parameters Avanzados

```
1 # Con diferentes tipos de datos:
2 from fastapi import FastAPI, Path
3 from enum import Enum
4
5 app = FastAPI()
6
7 # Usando Enum para opciones limitadas
8 class TipoUsuario(str, Enum):
9     admin = "admin"
10     cliente = "cliente"
11     moderador = "moderador"
12
13 @app.get("/usuarios/{tipo}/{id}")
14 async def obtener_usuario_por_tipo(
15     tipo: TipoUsuario,
16     id: int = Path(ge=1, le=9999, description="ID del usuario")
17 ):
18     return {
19         "tipo": tipo,
20         "usuario_id": id,
21         "mensaje": f"Usuario {tipo.value} con ID {id}"
22     }
```

```
1 # Con validaciones personalizadas:
2 @app.get("/productos/{codigo}")
3 async def obtener_producto(
4     codigo: str = Path(
5         min_length=3,
6         max_length=10,
7         regex="^[A-Z]{2}[0-9]{3,8}$",
8         description="Código de producto (ej: AB123)"
9     )
10 ):
11     return {"codigo_producto": codigo}
```

Trabajando con Query Parameters



```
1 @app.get("/productos")
2 async def obtener_productos(
3     categoria: str = "todas",
4     precio_max: float = 1000.0,
5     disponible: bool = True
6 ):
7     return {
8         "categoria": categoria,
9         "precio_max": precio_max,
10        "disponible": disponible,
11        "productos": ["producto1", "producto2"]
12    }
```

- **URLs de ejemplo:**

- '/productos' -> Usa los valores por defecto
- '/productos?categoria=electronics' -> Solo cambia categoría
- '/productos?categoria=ropa?precio_max=50' -> Múltiples parámetros
- '/productos?disponible=false' -> Filtrar por disponibilidad

- **Características:**

- **Opcionales:** Tienen valores por defecto
- **Flexibles:** Se pueden omitir o combinar
- **Automáticos:** FastAPI los detecta automáticamente

Query Parameters con Validación

```
1 from fastapi import Query
2 from typing import Optional, List
3
4 @app.get("/productos")
5 async def buscar_productos(
6     q: Optional[str] = Query(None, min_length=3, max_length=50, description="Término de búsqueda"),
7     categoria: str = Query("todas", description="Categoría del producto"),
8     precio_min: float = Query(0, ge=0, description="Precio mínimo"),
9     precio_max: float = Query(1000, gt=0, description="Precio máximo"),
10    limite: int = Query(10, ge=1, le=100, description="Número máximo de resultados"),
11    tags: List[str] = Query([], description="Lista de etiquetas")
12 ):
13     return {
14         "busqueda": q,
15         "categoria": categoria,
16         "precio_min": precio_min,
17         "precio_max": precio_max,
18         "limite": limite,
19         "tags": tags,
20         "resultados": ["producto1", "producto2"]
21     }
```

Combinando Ambos Tipos de Parámetros

```
1 from fastapi import FastAPI, Path, Query
2 from typing import Optional
3
4 @app.get("/tiendas/{tienda_id}/productos/{categoria}")
5 async def obtener_productos_tienda(
6     # Path parameters
7     tienda_id: int = Path(ge=1, description="ID de la tienda"),
8     categoria: str = Path(min_length=2, description="Categoría de productos"),
9     # Query parameters
10    precio_min: Optional[float] = Query(None, ge=0, description="Precio mínimo"),
11    precio_max: Optional[float] = Query(None, gt=0, description="Precio máximo"),
12    ordenar_por: str = Query("nombre", regex="^(nombre|precio|fecha)$", description="Campo por el que ordenar"),
13    orden: str = Query("asc", regex="^(asc|desc)$", description="Orden ascendente o descendente"),
14    pagina: int = Query(1, ge=1, description="Número de página"),
15    por_pagina: int = Query(20, ge=1, le=100, description="Productos por página")
16 ):
17     return {
18         "tienda_id": tienda_id,
19         "categoria": categoria,
20         "filtros": {
21             "precio_min": precio_min, "precio_max": precio_max,
22             "ordenar_por": ordenar_por, "orden": orden
23         },
24         "paginacion": {"pagina": pagina, "por_pagina": por_pagina},
25         "productos": []
26     }
```

Documentación Interactiva Automática

- **FastAPI genera automáticamente:**
 - **Swagger UI:** '/docs' - Interfaz interactiva para probar la API
 - **ReDoc:** '/redoc' - Documentación alternativa más detallada
 - **Esquema OpenAPI:** '/openapi.json' - Especificación JSON
- **¿Qué incluye la documentación?**
 - Todas las rutas y métodos HTTP
 - Parámetros de entrada y salida
 - Tipos de datos y validaciones
 - Ejemplos de requests y responses
 - Descripciones de cada endpoint

```
1 app = FastAPI(  
2     title="Mi API de E-commerce",  
3     description="API para gestionar productos y usuarios",  
4     version="1.0.0",  
5     contact={  
6         "name": "Equipo de desarrollo",  
7         "email": "dev@miempresa.com"  
8     },  
9     license_info={  
10        "name": "MIT",  
11    }  
12 )
```

Manejo de Errores en FastAPI

HTTPException para errores controlados

Respuestas de error personalizadas

```
1 from fastapi import HTTPException, status
2
3 @app.get("/usuarios/{user_id}")
4 async def obtener_usuario(user_id: int):
5     if user_id not in usuarios_db:
6         raise HTTPException(
7             status_code=status.HTTP_404_NOT_FOUND,
8             detail="Usuario no encontrado"
9         )
10    return usuarios_db[user_id]
```

```
1 @app.post("/usuarios", status_code=status.HTTP_201_CREATED)
2 async def crear_usuario(usuario: UsuarioCreate):
3     if usuario.email in emails_existentes:
4         raise HTTPException(
5             status_code=status.HTTP_400_BAD_REQUEST,
6             detail="El email ya está registrado"
7         )
8     # Crear usuario...
9     return nuevo_usuario
```


kingscorner